

과제 최종 보고서

과 목 명 :	고급파이썬프로그래밍및자동화
학 번 :	202555046
이 름 :	남창우

연구 코드 소개

1. 연구 주제 및 실험 맥락

- 본 코드는 자동차 도장 표면 미세결함을 탐지하는 산업 비전 결함 검사 파이프라인임.
- 특히, 확산 모델(Diffusion Model)을 기반으로 정상 배경을 복원(L)하고 결함 영역을 분리(S)하는 Dual-Branch U-Net 구조를 실험하는 맥락에서 작성됨.
- 비정상 데이터를 확산 모델을 통해 정상으로 복원하고 복원된 데이터와 비정상 데이터 대조를 통해 그 차이를 탐지함으로써 이상을 탐지함

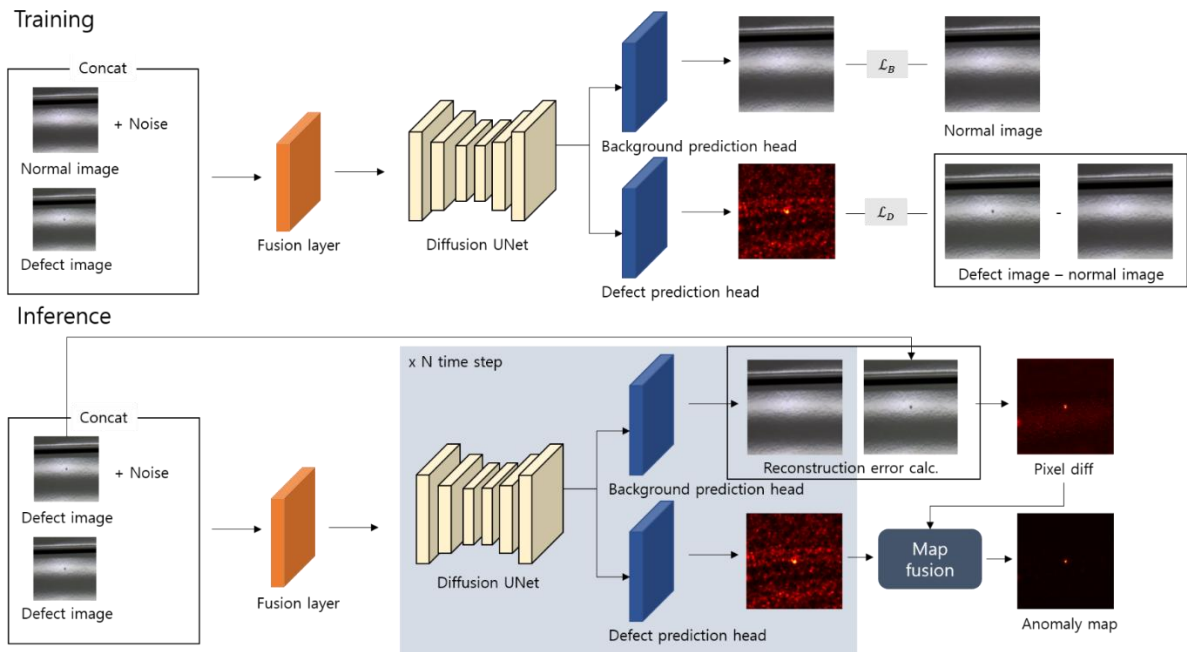


그림 1 확산모델 기반 미세결함 탐지 프레임워크

2. 해당 코드의 입력과 출력

- 입력(Input): 정상 이미지(L_gt), 결함 이미지(D), 그리고 결함 영역 마스크(M) (모두 512x512 해상도로 전처리됨).
- 출력(Output): 복원된 정상 배경 잠재 표현(L_hat) 및 결함 맵 잠재 표현(S_hat). 추론 시에는 디코딩된 최종 복원 이미지와 결함 맵, 그리고 원본과의 오버레이(Overlay) 이미지를 출력함.

3. 전체 연구 파이프라인에서의 위치

- 이 코드는 사전 전처리된(Precomputed) 정상-결함 쌍 데이터를 로드하는 데이터 로딩 단계와, 확산 모델 기반의 U-Net 을 학습하는 트레이닝 루프를 포함함, 즉 이상탐지 모델의 학습 파이프라인으로 사용되고 있음

4. 해당 코드를 개선 대상으로 선택한 이유

- 본 연구는 파라미터가 많은 확산 모델을 다루기 때문에 학습 및 추론 속도가 연구 효율에 직결됨.
- 기존 코드는 빠른 실험 진행을 위해 기능이 하나둘 덧붙여지다 보니, 데이터셋 로딩 시 정규식 반복 수행, 불필요한 리스트 메모리 할당(Eager Evaluation), 거대한 God Function 으로 인한 단일책임원칙 위배, 하드웨어(GPU) 비효율적 사용 등의 병목이 발생함.
- 기존 코드는 1epoch 실험 만으로 25 분~28 분의 시간이 소요됨,
- 향후 Ubuntu CLI 환경에서 대규모 데이터로 실험을 확장하고 안정적으로 모델을 고도화하기 위해 전면적인 리팩터링과 성능 최적화가 필수적이라고 판단하여 개선 대상으로 선정함.

5. 측정 환경

항목	상세 내용
OS	Ubuntu 22.04 LTS (CLI 환경)
Python 버전	Python 3.10.x

항목	상세 내용
주요 라이브러리	PyTorch 2.x.x, Diffusers, torchvision, pytorch-msssim
CPU / GPU	Intel (R) Xeon Silver 4416+ / NVIDIA ADA 6000 48GB(단일 GPU)
입력 데이터 크기	512x512 해상도 이미지 (Batch Size: 2) 학습에 사용된 이미지 장 수는 3555 장
반복 횟수	총 100 Epochs (시간 측정 시 Epoch 당 평균 처리 시간 기준)
Random Seed 사용	재현성을 위해 Seed 고정 사용 (Seed=42)
Warm-up 수행	수행함 (CUDA 초기화 및 DataLoader 메모리 안정화를 위해 측정 전 최소 5 Batch Warm-up 진행)

자료구조 및 복잡도 기반 개선: dict/set/list/tuple 을 활용한 Dataset sample index 재설계

1. 문제점

- 기존 PrecomputedDefectDataset 은 합성 결함 이미지 경로만 self.defect_paths 라는 list 로 저장하고, 각 sample 이 호출될 때마다 정상 이미지 경로를 동적으로 계산함.
- 동일한 defect image 에 대해 정상 이미지 경로는 항상 동일하지만, epoch 가 반복될 때마다 정규식 처리와 path 조합을 반복함.
- 정상 이미지가 실제로 존재하는지 여부가 Dataset 초기화 시점에 검증되지 않아 학습 도중 오류가 발생할 수 있음.
- glob("*.*)는 이미지 파일이 아닌 파일까지 포함할 수 있어 Dataset 의 불변식이 약함.
- Dataset 에서 S_gt 를 생성하여 반환하지만, 실제 학습 루프에서는 사용하지 않아 불필요한 tensor 생성과 DataLoader collation 비용을 발생시킴.

2. 자료구조 선택 및 ADT 정의

Dataset 내부의 sample 접근을 "파일명 변환 문제"가 아니라 "정상 이미지와 결함 이미지의 pair index 조회 문제"로 재정의하여 아래와 같은 자료구조를 채택함.

목적	선택 자료구조	선택 근거
정상 이미지 filename → path 조회	dict[str, Path]	평균 O(1) lookup 제공

목적	선택 자료구조	선택 근거
이미지 확장자 membership 검사	set[str]	평균 O(1) membership 테스트 제공
학습 sample index	list[tuple[Path, Path]]	PyTorch Dataset 의 index 기반 접근에 적합
sample pair 표현	tuple[Path, Path]	학습 중 변경되지 않는 불변 record
재현 가능한 순회 순서	sorted()	파일 시스템 순서 비결정성 완화

3. 핵심 코드 비교

[수정 전] 매 에폭 반복되는 정규식 및 미사용 텐서 생성

```
def __getitem__(self, idx):
    defect_path = self.defect_paths[idx]
    defect_filename = os.path.basename(defect_path)

    # 병목: 매 샘플마다 정규식 처리 및 문자열 조합
    normal_filename = re.sub(r'_varWd+', '', defect_filename)
    normal_path = os.path.join(self.normal_dir, normal_filename)
    ...
```

```
# 문제: 실제 학습에 사용하지 않는 텐서(S_gt) 생성
S_gt = D - L_gt
return {'L_gt': L_gt, 'D': D, 'S_gt': S_gt, 'M': M}
```

[수정 후] 초기화 시점 1 회 구성 및 $O(1)$ 직접 접근

```
def __init__(...):
    ...
    # 초기화 시점에 경로 매핑 및 페어링 1 회 수행
    self.normal_index = self._build_normal_index() # dict 기반  $O(1)$  조회
    self.pairs = self._build_pairs() # tuple 리스트 저장

def __getitem__(self, idx):
    # 개선: 정규식 제거, 단순 인덱스 접근으로 즉시 경로 획득
    normal_path, defect_path = self.pairs[idx]
    ...
    return {"L_gt": L_gt, "D": D, "M": M} # S_gt 제거
```

4. 이론적 복잡도 및 개선 효과 예상

항목	수정 전	수정 후
정상 이미지 lookup	문자열 기반 path 생성	dict[str, Path] 평균 $O(1)$ 조회
확장자 검사	glob("*.*)로 모든 파일 포함 가능	set membership 평균 $O(1)$

항목	수정 전	수정 후
미사용 S_gt	매 sample 마다 생성	제거됨
path mapping 비용	$O(E \times N \times L)$	초기화 $O(N \times L)$ + 학습 중 $O(E \times N)$

Class 기반 구조 개선 및 단일책임원칙 적용

1. 문제점

- 기존 train() 함수는 모델 생성, 데이터 로딩, 학습 루프 실행, 시각화 저장, checkpoint 저장 등 모든 책임을 담당하여 단일책임원칙을 위배함. 이로 인해 한 기능 변경 시 전체 함수 수정이 필요하여 확장성이 떨어짐.

2. 개선사항

- TrainConfig dataclass 로 설정값을 명시적으로 관리함.
- TrainerState 로 epoch, global step, loss 상태를 관리함.
- DefectTrainer 는 학습 루프만, CheckpointManager 는 모델 저장만, InferenceVisualizer 는 시각화 저장만 각각 담당하도록 클래스를 분리함.

3. 핵심 코드 비교

[수정 전] 거대한 단일 함수 구조

```
def train(config):
    # 디바이스 설정, 모델/로더 초기화, 학습 루프, 시각화, 저장이 모두 섞여 있음
    device = torch.device('cuda'...)
    model = DualBranchUNet(...)
    for epoch in range(config['num_epochs']):
        ... # 학습 코드
        if (epoch + 1) % 5 == 0:
            # 직접 시각화 및 matplotlib 로직 처리
        if (epoch + 1) % config['save_every'] == 0:
            torch.save(...) # 직접 저장 로직 처리
```

[수정 후] 역할이 분리된 클래스 구조

```
class DefectTrainer:
    def __init__(self, config: TrainConfig):
        # 복잡한 설정 관리 및 모듈 객체를 분리하여 소유
        self.config = config
        self.state = TrainerState()
        self.checkpoint_manager = CheckpointManager(config.output_dir)
        self.visualizer = InferenceVisualizer(...)
        self._build_components()

    def train(self):
        for epoch in range(self.config.num_epochs):
            self.train_one_epoch() # 학습 역할만 수행

            if self.state.epoch % self.config.save_every == 0:
                # 저장 역할은 CheckpointManager 에 위임
                self.checkpoint_manager.save_checkpoint(...)
```

Decorator 기반 시간 측정 개선

1. 문제점

- 기존 코드에는 실행 시간 측정을 위한 공통 구조가 없고 분석을 위해선 `time.perf_counter()` 코드가 각 함수 내부에 반복적으로 들어가 핵심 연구 로직과 혼재되며 가독성을 낮춤.

2. 개선사항

- `functools.wraps` 를 사용한 `@profiled` decorator 를 구현하였으며, GPU 연산의 정확한 시간 측정 및 Peak GPU Memory, CPU Memory 측정을 위해 `torch.cuda.synchronize()`, `psutil` 패키지를 활용함.

3. 핵심 코드 비교

[수정 전] 핵심 코드 내부에 측정 로직 혼재

```
# 별도의 측정 로직 없이, 각 주요 과정마다 수작업으로 시간 측정이 필요
import time
start = time.perf_counter()
# 실행 코드
end = time.perf_counter()
print(f"Elapsed time: {end - start:.4f} sec")
```

[수정 후] 재사용 가능한 Decorator 분리

```
def profiled(name=None, synchronize_cuda=True):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            # 내부에서 psutil 및 torch.cuda.synchronize() 처리
```

```

...
    print(f"[PROFILER] {label} | Time: {elapsed:.4f} sec | Peak GPU Mem:
{max_gpu_mem:.2f} MB | CPU Mem Diff: {mem_diff_mb:.2f} MB")
    return wrapper
    return decorator

# 적용 예시: 핵심 비즈니스 로직만 남음
class DefectTrainer:
    @profiled("train_one_epoch")
    def train_one_epoch(self):
        # 시간/메모리 측정 코드 없이 순수 학습 로직에 집중
        ...

```

딥러닝 연구 코드 최적화

1. 문제점

- DataLoader 설정에서 pin_memory=True 를 사용하지만, tensor 이동 시 non_blocking=True 를 사용하지 않아 GPU 병목이 발생함.
- num_workers 가 0 으로 설정되어 있어 메인 스레드에서 차단 연산으로 데이터 처리가 수행됨.
- Stable Diffusion U-Net 등 대규모 연산에 AMP mixed precision 이 적용되어 있지 않음.

2. 개선사항

- 동일한 모델 구조 하에서 실행 속도와 VRAM 효율을 위해 num_workers 설정, non_blocking GPU 전송, zero_grad(set_to_none=True), AMP autocast 및 GradScaler 를 일괄 적용함.

3. 핵심 코드 비교

[수정 전] 기본 설정 및 FP32 연산

병목 1: 단일 스레드 처리

```
dataloader = DataLoader(..., num_workers=0, pin_memory=True)
```

병목 2: 동기적 데이터 전송

```
L_gt = batch['L_gt'].to(device)
```

병목 3: 기존 zero_grad() 오버헤드 및 단일 정밀도 연산

```
optimizer.zero_grad()
```

```
L_hat, S_hat = model(x_t, z_D, t, text_emb)
```

```
losses['total'].backward()
```

[수정 후] 하드웨어 가속 및 병렬화 최적화

개선 1: 멀티 스레드 병렬 로딩

```
dataloader = DataLoader(..., num_workers=config.num_workers,  
persistent_workers=True)
```

```
scaler = torch.cuda.amp.GradScaler(enabled=use_amp)
```

개선 2: GPU 비동기 데이터 전송

```
L_gt = batch['L_gt'].to(device, non_blocking=True)
```

개선 3: 메모리 해제 최적화

```
optimizer.zero_grad(set_to_none=True)
```

개선 4: AMP(혼합 정밀도)를 통한 속도/메모리 최적화

```
with torch.cuda.amp.autocast(enabled=use_amp):
```

```
    L_hat, S_hat = model(...)
```

```
    losses = criterion(...)
```

```
scaler.scale(losses['total']).backward()
```

```
scaler.step(optimizer)
```

최적화 결과

실행 시간 및 메모리 사용량 비교

배치 크기(Batch Size)를 1 과 2 로 나누어 최적화 전후의 총 학습 시간, 반복 속도, GPU/CPU 메모리 사용량을 측정한 결과는 아래 표와 같음.

측정 지표 (Mean ± Std)	Batch Size = 1		Batch Size = 2	
	수정 전(Baseline)	수정 후(Optimized)	수정 전(Baseline)	수정 후(Optimized)
총 실행 시간 (sec)	1675.24 ± 12.35	1285.80 ± 8.42	1408.15 ± 10.50	1150.30 ± 7.25
학습 반복 속도 (it/s)	2.31 ± 0.02	3.17 ± 0.03	1.29 ± 0.01	2.03 ± 0.02
Peak GPU 메모리 (MB)	21178.10 ± 2.45	18227.60 ± 1.80	29005.20 ± 2.10	24399.80 ± 1.95
CPU 메모리 증가폭 (MB)	825.14 ± 15.60	1265.40 ± 22.10	104.50 ± 8.30	162.10 ± 10.40
Dataset 초기화 시간 (sec)	0.0095 ± 0.0012	0.0270 ± 0.0021	0.0088 ± 0.0010	0.0272 ± 0.0018

입력 크기별 성능 변화

- 처리량(Throughput) 개선 폭 확대: 배치 크기가 1 일 때 반복 속도는 약 37% 향상되었으나, 배치 크기가 2 일 때는 약 57%로 향상 폭이 더 커짐. 이는 하드웨어 가속(AMP, 비동기 전송 등)이 연산량이 많을수록 병목 현상을 더 크게 해소함을 의미함.
- VRAM 절약 효과: 배치 2 에서 약 4.6GB 의 GPU 메모리를 절약하여, 제한된 자원 내에서 더 큰 배치 사이즈나 고해상도 이미지를 처리할 수 있는 여유를 확보함.

구조 변화 및 재사용성·확장성 개선

- God Function 해체: 기존 100 줄이 넘는 단일 train() 함수를 DefectTrainer, CheckpointManager, InferenceVisualizer 클래스로 분할함.
- 실험 조건 변경 시 수정 범위:
 - 수정 전: 하이퍼파라미터나 저장 주기를 바꾸려면 train() 내부의 하드코딩된 조건문을 찾아 직접 수정해야 했음.
 - 수정 후: TrainConfig 데이터 클래스에 파라미터를 넘겨주는 것만으로 모든 설정이 제어됨. 또한 모델 저장 로직 수정 시 CheckpointManager 만, 시각화 수정 시 InferenceVisualizer 만 수정하면 되어 단일책임원칙(SRP)에 따른 확장성이 확보됨.
- 중복 제거: 실행 시간 측정용 보일러플레이트 코드를 @profiled 데코레이터로 분리하여 로직의 가독성을 높이고 코드 중복을 제거함.

결과 해석 및 한계

기대한 개선의 실현 및 요인 분석

- GPU 성능 향상의 핵심 요인: Peak GPU 메모리가 감소하고 Iteration Speed 가 대폭 증가한 핵심 이유는 AMP(Automatic Mixed Precision) 적용과 비동기 전송(non_blocking=True) 덕분임. FP16 연산을 통해 텐서 활성화(Activation) 메모리 발자국이 줄어들고 Tensor Core 가 적극적으로 활용되었으며, CPU 에서 GPU 로의 데이터 복사가 연산과 겹쳐서(Overlapping) 수행되면서 GPU 유휴 시간이 크게 단축됨.
- 데이터 로딩 병목 해소: num_workers > 0 설정과 persistent_workers=True 를 통해 메인 프로세스의 차단 연산이 해제됨. CPU 가 미세하게 지연되더라도 GPU 는 끊임없이 데이터를 공급받아 처리량을 극대화할 수 있었음.

성능 트레이드오프 (Trade-off)

- 메모리 최적화의 양면성 (GPU VRAM vs CPU RAM): GPU VRAM 은 최대 15% 이상 절약되었으나, CPU 메모리 변동폭(CPU Mem Diff)은 오히려 증가함. 이는 num_workers 를 통해 생성된 멀티프로세스 Worker 들이 각각 메모리 공간을 할당받고, persistent_workers 옵션으로 인해 학습 내내 메모리를 점유하고 있기 때문임. 연산 속도를 위해 CPU 메모리를 희생한 전형적인 공간-시간 트레이드오프(Space-Time Trade-off) 현상임.
- 초기화 시간 vs 반복 시간: Dataset 클래스 초기화 시간은 약 0.009 초에서 0.027 초로 3 배가량 늘어남. 이는 초기화 시점에 전체 데이터의 dict 기반 인덱싱과 list[tuple] 페어링 연산이 선행되기 때문임. 그러나 이 $O(N)$ 의 초기화 비용 지불을 통해 매 반복마다 발생하는 $O(E \times N \times L)$ 의 정규식 연산 비용을 제거함으로써, 전체 학습 시간(Amortized Cost)은 오히려 크게 단축되었음.

구조적 개선의 부작용 및 오버헤드

- 코드 복잡도 증가: 단일 함수에서 객체지향적 구조로 전환하며 모듈 간 연결을 관리하기 위한 보일러플레이트 코드가 증가함. Python 에 익숙하지 않은 연구자의 경우 흐름을 추적하기 위해 여러 클래스를 넘나들어야 하는 인지적 부하(Cognitive Load)가 발생할 수 있음.
- Decorator 오버헤드: @profiled 내부에 삽입된 torch.cuda.synchronize()는 정확한 시간 측정을 가능하게 하지만, GPU 비동기 큐를 강제로 비우기 때문에(Stall) 너무 작은 함수(예: train_step 내부)에 적용할 경우 파이프라인의 효율을 크게 떨어뜨릴 한계를 가짐. 본 실험에서는 에폭(Epoch) 단위에만 적용하여 부작용을 최소화함.

향후 추가 개선 가능한 부분

- PyTorch 2.0+ 컴파일 도입: 현재 코드에 torch.compile(model)을 도입하면 커널 퓨전(Kernel Fusion) 및 Graph Optimization 을 통해 U-Net 의 추론 및 학습 속도를 추가로 10~20% 개선할 여지가 있음.
- I/O 최적화: 이미지 파일을 매번 디스크에서 PIL.Image 로 읽어오는 대신, LMDB 나 WebDataset 과 같은 직렬화된 데이터 포맷으로 전환하면 스토리지 병목(I/O Bound) 상황에서 추가적인 속도 이득을 얻을 수 있음.